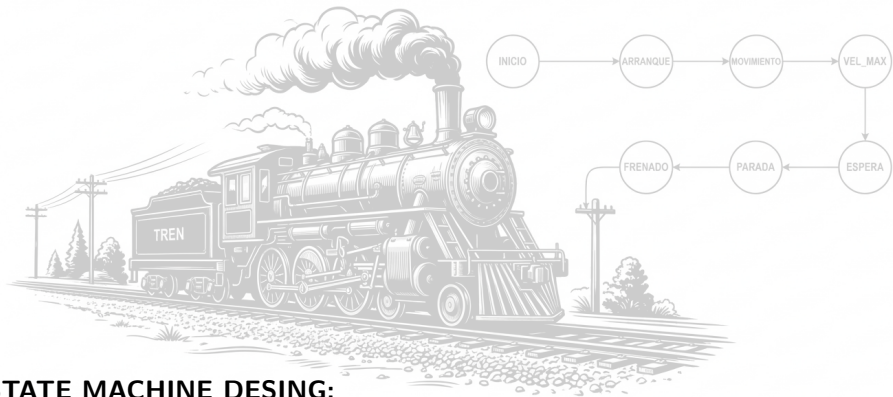




STATE MACHINE DESING: THE ELECTRIC TRAIN CONTROLLER

Jesús Tovar T00085106

Joiber Gutierrez T00063678

Universidad Tecnológica de Bolívar
25 de abril de 2026



¿Qué es una Máquina de Estado Finita (FSM)?

Definición y Conceptos Fundamentales

Representa un sistema como un conjunto de **estados** y **transiciones** que dependen de las entradas, generando salidas asociadas. Es la representación de un circuito secuencial particular.

- **Estado:** Conjunto único de valores medidos en un sistema digital.
- **Ramificación:** El paso del estado presente al estado siguiente.
- **Diagrama de estado:** Ilustración del funcionamiento mediante "burbujas" y "flechas".
- **Importancia:** Cualquier circuito con memoria (desde un flip-flop hasta una computadora) puede ser considerado una FSM.

State Machine Design: The Electric Train Controller



Planteamiento del Problema

Se tiene un sistema de trenes eléctricos compuesto por dos trenes, denominados **Tren A** y **Tren B**, que se desplazan en un circuito con múltiples segmentos de vía interconectados.

El sistema incluye un **segmento común (Track 2)**, el cual puede ser ocupado por ambos trenes, lo que introduce el riesgo de **colisiones** si no se controla adecuadamente su acceso.



Objetivo del Controlador

Diseño de la FSM

Diseñar una **máquina de estados finita (FSM)** que:

- Controle el movimiento de ambos trenes mediante señales de dirección.
- Gestione el estado de los **switches** para enrutar correctamente los trenes.
- Garantice que **solo un tren ocupe el segmento común** en cualquier instante.
- Evite colisiones y descarrilamientos durante la operación del sistema.

Introducción

Contexto General

Las **máquinas de estados finitas (FSM)** constituyen una herramienta fundamental en el diseño de sistemas digitales secuenciales, permitiendo modelar comportamientos complejos mediante la definición de estados y transiciones controladas por entradas.

En aplicaciones reales, como sistemas de control y automatización, las FSM son ampliamente utilizadas para garantizar un funcionamiento seguro, predecible y eficiente.

Aplicación al Problema

En este trabajo, se aborda el diseño de un controlador para un sistema de trenes eléctricos, en el cual dos trenes comparten parcialmente la vía. Mediante el uso de una FSM, se busca gestionar el movimiento de los trenes y el estado de los switches, asegurando una operación libre de colisiones y respetando las restricciones del sistema físico.

Objetivos



Objetivo General

Diseñar e implementar una **máquina de estados finita (FSM)** en VHDL para el control de un sistema de trenes eléctricos, garantizando una operación segura, libre de colisiones y acorde a las condiciones establecidas en el problema.

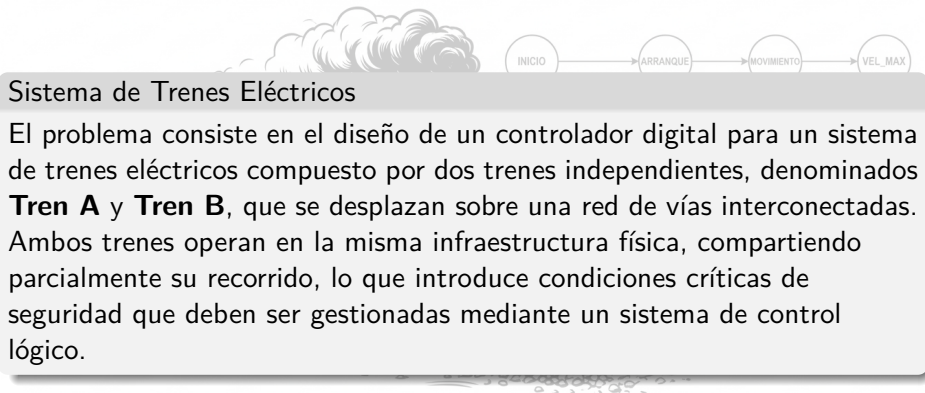
Objetivos Específicos

Metas del Proyecto

- Analizar el funcionamiento del sistema de trenes y sus restricciones operativas.
- Identificar las **entradas y salidas** del sistema a partir de los sensores y actuadores.
- Diseñar el **diagrama de estados** que modele el comportamiento del controlador.
- Implementar la FSM utilizando **VHDL** en la plataforma Quartus.
- Verificar el funcionamiento del sistema mediante **simulación** y análisis del RTL.
- Documentar el desarrollo del proyecto y su implementación en un repositorio.



Descripción del Problema



Sistema de Trenes Eléctricos

El problema consiste en el diseño de un controlador digital para un sistema de trenes eléctricos compuesto por dos trenes independientes, denominados **Tren A** y **Tren B**, que se desplazan sobre una red de vías interconectadas. Ambos trenes operan en la misma infraestructura física, compartiendo parcialmente su recorrido, lo que introduce condiciones críticas de seguridad que deben ser gestionadas mediante un sistema de control lógico.

Estructura del Sistema

Segmentos de la Vía

La red de trenes está dividida en múltiples segmentos, entre los cuales destaca un **segmento común (Track 2)**, el cual puede ser accedido por ambos trenes.

Este segmento representa un punto crítico del sistema, ya que:

- Es compartido por ambos trenes.
- Existe riesgo de **colisión** si no se regula su acceso.
- Requiere coordinación estricta mediante lógica de control.

Detección de Trenes

El sistema dispone de cinco sensores digitales (**S1, S2, S3, S4, S5**) ubicados en puntos estratégicos de la vía, los cuales permiten detectar la presencia de los trenes.

- Cada sensor se activa cuando un tren se encuentra en su proximidad.
- Las señales de los sensores **no son instantáneas**, sino que permanecen activas durante varios ciclos de reloj.
- Esto implica que el diseño debe considerar **transiciones de estado cuidadosas** para evitar interpretaciones erróneas.

Señales de Control del Sistema

Actuadores del Sistema

El controlador genera señales de salida que determinan el comportamiento del sistema:

- **Dirección de los trenes:**
 - Tren A: DA1-DA0
 - Tren B: DB1-DB0
- **Configuración de switches:** SW1, SW2 y SW3

Estas señales permiten:

- Detener o poner en movimiento los trenes.
- Controlar la dirección de desplazamiento.
- Redirigir los trenes entre vías internas y externas.

Uso de Máquina de Estados

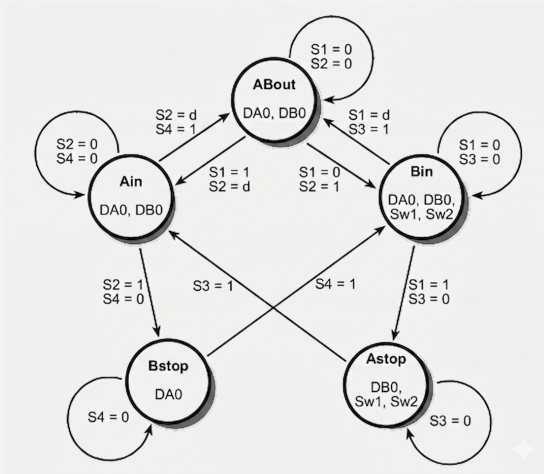
Para resolver este problema, se emplea una **máquina de estados finita (FSM)**, la cual permite modelar el comportamiento del sistema mediante estados y transiciones basadas en las señales de entrada.

La FSM:

- Supervisa continuamente los sensores.
- Decide cuándo permitir el acceso al segmento común.
- Controla la detención y movimiento de los trenes.
- Garantiza el cumplimiento de todas las restricciones del sistema.

Representación Visual: Diagrama de Estados

Diagrama de Estados del Controlador de Trenes



Análisis del Diagrama de Estados

Interpretación del Comportamiento

- La máquina de estados está compuesta por cinco estados principales: **ABout**, **Ain**, **Bin**, **Astop** y **Bstop**.
- Las transiciones dependen de las combinaciones de los sensores (**S1**, **S2**, **S3**, **S4**).
- Los estados **Ain** y **Bin** representan el ingreso al segmento común.
- Los estados **Astop** y **Bstop** detienen un tren para evitar conflictos en la vía.
- El estado **ABout** representa la operación normal sin conflicto en el segmento compartido.

Descripción del Código Desarrollado

Estructura General

El controlador del sistema de trenes fue implementado en **VHDL** utilizando una arquitectura basada en **máquinas de estados finitas (FSM)**.

El diseño se compone de:

- Definición de la entidad (entradas y salidas)
- Declaración de estados y señales internas
- Proceso secuencial para la transición de estados
- Lógica combinacional para definir las salidas

Este enfoque permite modelar el comportamiento del sistema de forma clara, modular y sintetizable en hardware.

Definición de Entradas y Salidas

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.STD_LOGIC_ARITH.ALL;
USE IEEE.STD_LOGIC_UNSIGNED.ALL;

ENTITY Tcontrol IS
PORT(
    reset, clock,
    sensor1, sensor2, sensor3, sensor4, sensor5 : IN STD_LOGIC;
    switch1, switch2, switch3 : OUT STD_LOGIC;
    dirA, dirB : OUT STD_LOGIC_VECTOR(1 DOWNTO 0)
);
END Tcontrol;
```



Definición de Estados

Tipo Enumerado y Señales

```
ARCHITECTURE a OF Tcontrol IS
```

```
TYPE STATE_TYPE IS (ABout, Ain, Bin, Astop, Bstop);
```

```
SIGNAL state : STATE_TYPE;
```

```
SIGNAL sensor12, sensor13, sensor24 :
```

```
STD_LOGIC_VECTOR(1 DOWNT0 0);
```

```
BEGIN
```

Interpretación

- Se utiliza un tipo enumerado para representar los estados.
- Se definen combinaciones de sensores para simplificar decisiones.

Proceso Secuencial

Transición de Estados

```
PROCESS (reset, clock)
BEGIN
  IF reset = '1' THEN
    state <= ABout;

  ELSIF rising_edge(clock) THEN

    CASE state IS
```

Interpretación

- Se implementa un registro de estado con flip-flops.
- El cambio de estado ocurre en el flanco positivo del reloj.
- Se define un estado inicial mediante reset.

en entra al track común

res S1 y S2 simultaneamente.

Estados de Control

Astop y Bstop

```
WHEN Astop =>  
  IF sensor3 = '1' THEN  
    state <= Ain;  
  ELSE  
    state <= Astop;  
  END IF;
```

```
WHEN Bstop =>  
  IF sensor4 = '1' THEN  
    state <= Bin;  
  ELSE  
    state <= Bstop;  
  END IF;
```

Interpretación

- Se detiene un tren hasta que el otro libere el track común.
- Se usan sensores para reanudar el movimiento.

Procesamiento de Sensores



Agrupación de Entradas

```
sensor12 <= sensor1 & sensor2;  
sensor13 <= sensor1 & sensor3;  
sensor24 <= sensor2 & sensor4;
```

Interpretación

- Se combinan sensores para simplificar la lógica.
- Permite evaluar múltiples condiciones en una sola comparación.



WITH SELECT

- dependen únicamente del estado actual.
 es una máquina de tipo Moore.

Control de Movimiento

Dirección de los Trenes

```
WITH state SELECT
dirA <= "01" WHEN ABout,
      "01" WHEN Ain,
      "01" WHEN Bin,
      "00" WHEN Astop,
      "01" WHEN Bstop;
```

```
WITH state SELECT
dirB <= "01" WHEN ABout,
      "01" WHEN Ain,
      "01" WHEN Bin,
      "01" WHEN Astop,
      "00" WHEN Bstop;
```

Interpretación

- "01" indica movimiento hacia adelante.
- "00" indica detención del tren.
- Cada estado define el comportamiento de ambos trenes.

RTL del Sistema

Vista General del RTL

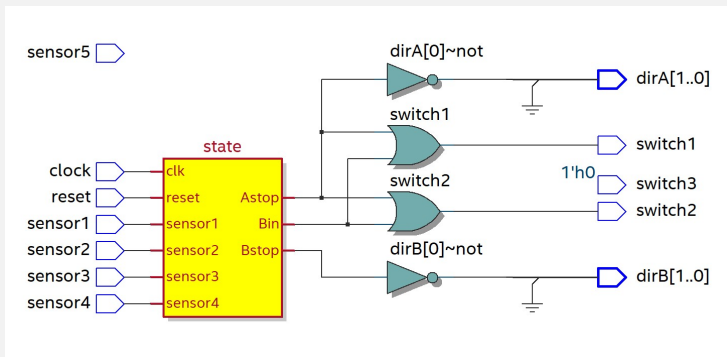


Diagrama de flujo de un proceso de aceleración de un vehículo:

```

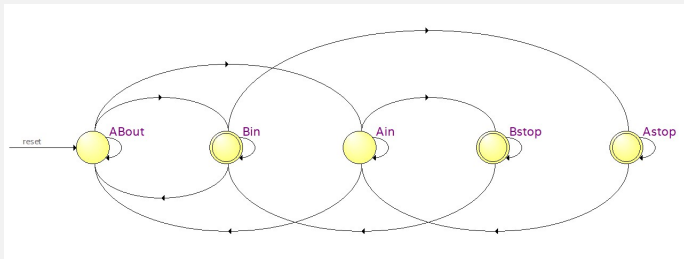
graph LR
    INICIO((INICIO)) --> ARRANQUE((ARRANQUE))
    ARRANQUE --> MOVIMIENTO((MOVIMIENTO))
    MOVIMIENTO --> VEL_MAX((VEL_MAX))
    VEL_MAX --> FIN((FIN))
  
```

El diagrama muestra una secuencia de cuatro estados: INICIO, ARRANQUE, MOVIMIENTO y VEL_MAX, conectados por flechas horizontales. A la izquierda del diagrama hay una ilustración de un coche acelerando, con humo saliendo del escape y una nube de polvo levantada por las ruedas traseras.

- El RTL representa la implementación hardware del diseño en VHDL.
- Se observan los bloques principales: lógica combinatorial y registros.
- El sistema está sincronizado por la señal de reloj.

RTL de la Máquina de Estados

Implementación de la FSM



Descripción

- Se observa el registro de estado implementado con flip-flops.
- La lógica de transición determina el siguiente estado.
- Las salidas dependen únicamente del estado actual (Modelo Moore).

Análisis del RTL

Interpretación del Diseño

- El diseño implementa correctamente una **máquina de estados finita**.
- El bloque secuencial almacena el estado actual del sistema.
- El bloque combinacional calcula el siguiente estado en función de los sensores.
- Las salidas se generan de manera estable, evitando cambios bruscos.
- El diseño es consistente con el diagrama de estados del problema.

Conclusión Técnica

La implementación en RTL valida que el código VHDL describe correctamente el comportamiento esperado del sistema, garantizando una operación segura y sincronizada.

Repositorio del Proyecto

Descripción General

El proyecto fue almacenado en un repositorio llamado:

PROBLEMA-DEL-TREN-PARCIAL-CORTE-2-DIGITALES-II

Este repositorio contiene todos los archivos necesarios para el desarrollo y validación del sistema:

- Código fuente en **VHDL**
- Archivos del proyecto en **Quartus**
- Imágenes del **RTL** y diagramas
- Presentación en **LaTeX**
- Video en **Youtube**

Conclusiones



Resultados Obtenidos

- Se logró diseñar e implementar una **máquina de estados finita (FSM)** capaz de controlar el movimiento de dos trenes evitando colisiones en el segmento común.
- El modelo desarrollado garantiza una operación segura mediante la correcta gestión de sensores y señales de control.
- La implementación en **VHDL** permitió describir el comportamiento del sistema de forma estructurada y sintetizable.

Conclusiones Finales

Análisis del Diseño

- El RTL generado confirmó la correcta correspondencia entre el diseño teórico y su implementación en hardware.
- El uso de una arquitectura tipo **Moore** permitió obtener salidas estables y predecibles.
- La simulación del sistema validó el comportamiento esperado en diferentes escenarios de operación.

El diseño cumple con los requerimientos del problema y demuestra la efectividad del uso de FSM en sistemas de control digital.

Simulación del Sistema



Configuración de la Simulación

Se realizó la simulación funcional del sistema en Quartus aplicando estímulos sobre las señales de entrada.

- Se utilizó una señal de reloj periódica.
- Se aplicó un pulso de **reset** para inicializar la FSM en el estado **ABout**.
- Se modificaron las entradas **sensor1** y **sensor2** para provocar transiciones de estado.

Resultados de la Simulación

Resultados Obtenidos

- Se observó la transición correcta desde **ABout** hacia **Bin**.
- Como resultado, las salidas **switch1** y **switch2** se activaron correctamente.
- Las señales de dirección **dirA** y **dirB** respondieron de acuerdo con el estado del sistema.
- No se presentaron valores indefinidos, evidenciando un diseño estable.

Evidencia

Las imágenes correspondientes a la simulación se encuentran disponibles en el repositorio del proyecto para su verificación.